哈尔滨工业大学　法国里昂商学院
Harbin Institute of Technology　emlyon business school

Object Oriented Programming
Python BootCamp

实战案例合集 / Practice

黄子扬

Ziyang Huang

二叉苹果树

June 13, 2026

Contents

【Example / 实战案例】Basic Assignment / 基础赋值

```python
age = 25
print(age)      # Output: 25
```

【Example / 实战案例】Valid and Invalid Names / 合法与非法命名

```python
# Valid names / 合法命名
test1 = 1
_alpha = 2
L = 3
car_length = 2000

# Invalid names / 非法命名
# 3text = 4          # Error: cannot start with digit
# my-var = 5         # Error: hyphen is not allowed
```

【Example / 实战案例】Comment Examples / 注释示例

```python
# This variable stores the car length in cm
L = 2000
print(L)

H = 146   # This variable stores the car height in cm
print(H)
```

【Example / 实战案例】Using type() / type() 实操

```python
num = 100               # An integer variable
length = 10.7           # A floating point variable

print(type(num))        # <class 'int'>
print(type(length))     # <class 'float'>
```

【Example / 实战案例】Boolean Examples / 布尔值示例

```python
is_finished = True
print(type(is_finished))    # <class 'bool'>

is_finished = False
print(type(is_finished))    # <class 'bool'>
```

【Example / 实战案例】Escape Character / 转义字符示例

```
text1 = 'She said, "What time is it?"'
text2 = "She said, \"What time is it?\""
print(text1)    # She said, "What time is it?"
print(text2)    # She said, "What time is it?"
```

【Example / 实战案例】Using len() / len() 函数

`len()` is a Python built-in function that returns the number of characters in a string (including spaces).

```
text2 = "This is a test string"
print(len(text2))   # Output: 22
```

【Example / 实战案例】Multiline String Examples / 多行字符串示例

```
# Method 1: Backslash / 反斜杠法
long_text = "This planet has--or rather had--a problem, which was \
this: most of the people living on it were unhappy for pretty much of the time."
print(long_text)

# Method 2: Triple quotes / 三引号法
long_text = """This planet has--or rather had--a problem, which was
this: most of the people living on it were unhappy for pretty much of the time."""
print(long_text)
```

【Example / 实战案例】Concatenation Example / 拼接示例

```
string1 = "Hello"
string2 = "World"
string3 = string1 + string2
print(string3)      # Output: HelloWorld (no space!)
```

【Example / 实战案例】Positive and Negative Indexing / 正索引与负索引

```
text = "Python BootCamp"

# Positive index / 正索引（从左到右，从 0 开始）
print(text[0])      # 'P'（第一个字符）
print(text[7])      # 'B'

# Negative index / 负索引（从右到左，从 -1 开始）
print(text[-1])     # 'p'（最后一个字符）
print(text[-5])     # 't'
```

【Example / 实战案例】Slicing Examples / 切片实战

```python
text = "Python BootCamp"

# Basic slicing [start:stop]
print(text[0:6])    # 'Python' (indices 0,1,2,3,4,5 — NOT 6)
print(text[7:11])   # 'Boot'

# Omit start: assumes 0
print(text[:6])     # 'Python' (same as text[0:6])

# Omit stop: goes to the end
print(text[7:])     # 'BootCamp'

# Both omitted: the whole string
print(text[:])      # 'Python BootCamp'
```

【Example / 实战案例】Immutability Demonstration / 不可变性演示

```python
name = "Emlyon"
name.upper()            # Returns "EMLYON" but does NOT change 'name'!
print(name)             # Still "Emlyon"

name = name.upper()     # Correct: reassign the result
print(name)             # Now "EMLYON"
```

【Example / 实战案例】Common String Methods / 常用字符串方法

```python
# 1. Case Conversion / 大小写转换
print("Python BootCamp".lower())        # 'python bootcamp'
print("Python BootCamp".upper())        # 'PYTHON BOOTCAMP'

# 2. Removing Whitespace / 去除首尾空白
print(" Python BootCamp".strip())       # 'Python BootCamp'

# 3. Counting Substrings / 统计子串出现次数
text = "Python BootCamp is the best Python program ever!"
print(text.count("Python"))             # Output: 2

# 4. Finding Substring Position / 查找子串位置
print(text.find("Python"))              # Output: 0 (first occurrence)
print(text.find("python"))              # Output: -1 (not found!)

# 5. Check Start and End / 检查首尾
print("Python BootCamp".startswith('P'))  # True
print("Python BootCamp".endswith('p'))    # False

# 6. Replace / 替换
print("Hello World".replace("World", "Python"))  # 'Hello Python'
```

【Example / 实战案例】None Example / None 示例

```
my_var = None
print(my_var)        # None
print(type(my_var)) # <class 'NoneType'>
```

【Example / 实战案例】Arithmetic Operators Demo / 算术运算符演示

```
print(2 + 2)         # 4
print(3 * 2)         # 6
print(3 ** 2)        # 9
print(5 / 2)         # 2.5 (standard division)
print(5 // 2)        # 2 (floor division)
```

【Example / 实战案例】String Arithmetic / 字符串运算

```
print('2' + '4')         # '24' (concatenation, not addition!)
print(2 * 'S')           # 'SS' (repetition)
# print('2' * '4')       # Error! Cannot multiply string by string
```

【Example / 实战案例】Comparison Examples / 比较运算示例

```
print(2.5 < 3)           # True
print(2 == 2)            # True
print(2 != 3)            # True
print(5 >= 5)            # True
print(10 < 3)            # False
```

【Example / 实战案例】Logical Operators Demo / 逻辑运算符演示

```
# or: True if either is True
print((2 == 3) or (4 < 5))       # True (4 < 5 is True)

# and: True only if BOTH are True
print((2 == 3) and (4 < 5))      # False (2 == 3 is False)

# not: Reverses the boolean
print(not (2 == 3))              # True (reverses False to True)
print(not True)                  # False
```

【Example / 实战案例】Type Casting Examples / 类型转换示例

```
print(int(9.9))          # 9 (truncates, does NOT round!)
print(str(5))            # '5'
```

```
print(bool(1))          # True
# print(int("Yes!"))    # ValueError: invalid literal for int()
```

【Example / 实战案例】Comma Method / 逗号法

```
name = "Doctor Strange"
heads = 2
arms = 3
print(name, "has", str(heads), "heads and", str(arms), "arms.")
# Output: Doctor Strange has 2 heads and 3 arms.
```

Note: `print()` automatically inserts spaces between comma-separated items. You must convert numbers to strings with `str()`.

注意：`print()` 会自动在逗号分隔的项之间插入空格。数字必须用 `str()` 转为字符串。

【Example / 实战案例】Concatenation Method / 拼接法

```
print(name + " has " + str(heads) + " heads and " + str(arms) + " arms.")
# Output: Doctor Strange has 2 heads and 3 arms.
```

Note: Tedious! Must manually add spaces and convert numbers to strings.

注意：这种方法写起来很繁琐！必须手动添加空格、用 `str()` 转换数字。

【Example / 实战案例】f-string Example / f-字符串示例

```
print(f"{name} has {heads} heads and {arms} arms.")
# Output: Doctor Strange has 2 heads and 3 arms.
```

【Example / 实战案例】.format() Method / .format() 方法示例

```
print("{} has {} heads and {} arms.".format(name, heads, arms))
# Output: Doctor Strange has 2 heads and 3 arms.
```

Curly braces `{}` act as placeholders, filled by `.format()` arguments in order.

【Example / 实战案例】% Operator / % 运算符示例

```
print("%s has %s heads and %s arms." % (name, heads, arms))
# Output: Doctor Strange has 2 heads and 3 arms.
```

`%s` is a placeholder for string; values are provided in parentheses after `%`.

【Example / 实战案例】input() Examples / 用户输入示例

```
# Basic usage
name = input()
```

```
print(name)

# With a prompt / 带提示信息的输入
age = input("Please enter your age: ")
print(age)              # age is a STRING, not a number!
print(type(age))        # <class 'str'>

# Convert to number if needed / 如需数字，必须转换
age = int(input("Please enter your age: "))
print(age + 1)          # Now it works as a number
```

【Example / 实战案例】Price Classification / 价格分级案例

课件中通过以下案例展示了完整的 `if-elif-else` 结构：

```
price = 42
if price > 100:
    print("expensive")          # Runs if price > 100
elif price > 50:                # Checked ONLY if price <= 100
    print("moderate")           # Runs if 50 < price <= 100
else:                           # Runs if all above are False
    print("cheap")              # Runs if price <= 50
# Output: cheap
```

【Example / 实战案例】Simple If-Statement / 简单 If 判断

课件展示了只需判断单个条件的场景：

```
name = "John"
if len(name) < 3:
    print("your name is short!")    # Won't run because len("John") = 4
```

如果条件为 `False`，缩进代码块什么都不执行。

【Example / 实战案例】Understanding len() / 理解 len() 的执行过程

```
num_letters = len("emlyon")
print(num_letters)          # Output: 6
```

What happens behind the scenes:

1. `len()` is called with the argument `"emlyon"`.
2. The length of the string is calculated —the number 6.
3. `len()` returns 6 and replaces the function call with the value.
4. Python then assigns `6` to `num_letters`. Equivalent to: `num_letters = 6`

【Example / 实战案例】Creating Your First Function / 创建你的第一个函数

```python
def multiply(x, y):          # Function signature: name + parameters
    product = x * y          # Function body (indented!)
    return product           # Return statement: sends result back

# Calling the function
result = multiply(2, 4)      # result = 8
print(result)                # 8
```

【Example / 实战案例】Scope Demonstration / 作用域演示

```python
x = 2                        # Global x / 全局变量

def func():
    x = 3                    # NEW local x — does NOT change global x!
    print("Inside:", x)      # Prints 3 (local x)

func()                       # Output: Inside: 3
print("Outside:", x)         # Output: 2 (global x unchanged!)
```

关键理解：函数内部的 `x = 3` 创建了一个**全新的局部变量**，不会影响全局的 `x`。

【Example / 实战案例】Scope Lookup Example / 作用域查找示例

```python
x = 5                        # Global

def func():
    y = x + 5                # x not found locally → uses global x (5)
    print("Inside, y =", y) # Output: Inside, y = 10

func()
print("Outside, x =", x)     # Output: Outside, x = 5
# print(y)                   # NameError! y only exists in func()'s local scope
```

注意：函数内部可以**读取**全局变量，但不能直接修改它（除非用 `global` 关键字）。

【Example / 实战案例】Printing Numbers with for / 用 for 循环打印数字

```python
for i in range(1, 11):
    print(i)                 # Prints 1, 2, 3, ..., 10 (each on new line)
```

其中 `i` 是迭代变量，依次取值为 1, 2, 3, ⋯, 10。

【Example / 实战案例】While Loop vs For Loop / While 循环与 For 循环对比

```python
# For loop version — finite, known iterations
for i in range(1, 11):
    print(i)
```

```

# While loop version — same result, different approach
counter = 1
while counter < 11:
    print(counter)
    counter = counter + 1       # MUST update counter!
```

【Example / 实战案例】Control Keywords Demo / 控制关键字对比

```
# Example: break — 立即终止整个循环
for i in range(1, 11):
    if i == 5:
        break           # Exit loop completely when i = 5
    print(i)            # Prints: 1, 2, 3, 4 (stops at 5)
```

```
# Example: continue — 跳过本次循环剩余部分
for i in range(1, 11):
    if i == 5:
        continue        # Skip the rest of THIS iteration
    print(i)            # Prints: 1, 2, 3, 4, 6, 7, 8, 9, 10 (no 5!)
```

```
# Example: pass — 占位符，什么都不做
for i in range(1, 11):
    if i == 5:
        pass            # Do nothing — just a placeholder
    print(i)            # Prints: 1, 2, 3, 4, 5, 6, 7, 8, 9, 10
```

【Example / 实战案例】print() with sep and end / print() 的 sep 和 end 参数

```
# Default behavior
print("hello", "there")
# Output: hello there (separated by space, ends with newline)

# Custom sep and end
print("hello", "there", sep="", end="!")
# Output: hellothere! (no separator, ends with "!" instead of newline)
```

【Example / 实战案例】Tuple Basics / 元组基础

```
tuple1 = (1, 2, 3)
print(type(tuple1))             # <class 'tuple'>

# Heterogeneous / 异构: 可以包含不同类型
tuple2 = (1, "John", "Red")
print(tuple2)                   # (1, 'John', 'Red')

# Empty tuple / 空元组
```

```
tuple3 = ()
print(len(tuple3))              # 0
```

【Example / 实战案例】Length, Indexing, Slicing / 长度、索引、切片

```
tuple2 = (1, "John", "Red")

# Length / 长度
print(len(tuple2))              # 3

# Indexing / 索引（和字符串一样！）
print(tuple2[1])                # 'John'

# Slicing / 切片
print(tuple2[1:3])              # ('John', 'Red')
```

【Example / 实战案例】The in Keyword / in 关键字

```
tuple2 = (1, "John", "Red")

if "Red" in tuple2:
    print('There is red')       # This runs
else:
    print('There is no red')
```

【Example / 实战案例】Tuples Are Iterable / 元组可迭代

```
tuple2 = (1, "John", "Red")
for item in tuple2:
    print(item)
# Output:
# 1
# John
# Red
```

【Example / 实战案例】Creating Lists / 创建列表

```
mylist1 = [1, 2, 3]
print(type(mylist1))            # <class 'list'>

# Like tuples, lists can contain different types
mylist2 = [1, "John", "Red"]
print(mylist2)                  # [1, 'John', 'Red']

# Create a list from string using split()
groceries = "eggs, milk, cheese"
```

```
grocery_list = groceries.split(", ")
print(grocery_list)              # ['eggs', 'milk', 'cheese']
```

【Example / 实战案例】Mutability Demo / 可变性演示

```
grocery_list = ['eggs', 'milk', 'cheese']
grocery_list[1] = 'butter'
print(grocery_list)              # ['eggs', 'butter', 'cheese']
```

【Example / 实战案例】insert(), pop(), append(), extend() / 四大核心方法

```
mylist2 = []
mylist2.insert(0, 'A')           # Insert 'A' at index 0
print(mylist2)                   # ['A']

mylist2.insert(2, 'C')           # Insert at index 2 (beyond end → appended)
print(mylist2)                   # ['A', 'C']

mylist2.insert(1, 'B')           # Insert 'B' at index 1
print(mylist2)                   # ['A', 'B', 'C']

# pop(i): Remove and RETURN the value at index i
out = mylist2.pop(2)             # Removes 'C'
print(out)                       # 'C'
print(mylist2)                   # ['A', 'B']

# append(x): Add to the end
mylist2.append('D')
print(mylist2)                   # ['A', 'B', 'D']

# extend(iterable): Add multiple elements to the end
mylist2.extend(['E', 'F'])
print(mylist2)                   # ['A', 'B', 'D', 'E', 'F']
```

【Example / 实战案例】sum(), min(), max() / 求和、最小值、最大值

```
mylist3 = [4, 17, 42, 5]

# Traditional way / 传统方式
total = 0
for number in mylist3:
    total = total + number
print(total)                     # 68

# Pythonic way / Python 风格
print(sum(mylist3))              # 68
print(min(mylist3))              # 4
print(max(mylist3))              # 42
```

【Example / 实战案例】Right Way to Copy / 正确拷贝方式

```python
animals = ["lion", "tiger"]

# WRONG way / 错误方式 — 只是别名
large_cats = animals
large_cats.append("Leopard")
print(large_cats)   # ['lion', 'tiger', 'Leopard']
print(animals)      # ['lion', 'tiger', 'Leopard'] — ALSO changed!

# RIGHT way / 正确方式 — 使用切片创建独立副本
animals = ["lion", "tiger"]
large_cats = animals[:]          # Creates a NEW list with same values
large_cats.append("Leopard")
print(large_cats)   # ['lion', 'tiger', 'Leopard']
print(animals)      # ['lion', 'tiger'] — unchanged!
```

【Example / 实战案例】Nested Structures / 嵌套示例

```python
my_nested_list = [[1, 2], [3, 4]]
print(my_nested_list)           # [[1, 2], [3, 4]]
print(my_nested_list[1])        # [3, 4]

# Double index notation / 双重索引
print(my_nested_list[1][0])     # 3 (first element of second sublist)
```

【Example / 实战案例】Sorting Examples / 排序示例

```python
# .sort() — in-place, list only
numbers = [1, 10, 5, 3]
numbers.sort()                  # Modifies 'numbers' directly
print(numbers)                  # [1, 3, 5, 10]

# sorted() — returns new list, works on any iterable
numbers = [1, 10, 5, 3]
new = sorted(numbers, reverse=True)
print(new)                      # [10, 5, 3, 1]
print(numbers)                  # [1, 10, 5, 3] — unchanged!

# Sort by key / 按自定义规则排序
colors = ["red", "yellow", "green", "blue"]
colors.sort(key=len)            # Sort by string length
print(colors)                   # ['red', 'blue', 'green', 'yellow']
```

【Example / 实战案例】Dictionary Basics / 字典基础操作

```python
# Creating a dictionary / 创建字典
capitals = {
    "France": "Paris",
    "Spain": "Madrid",
    "Italy": "Rome",
}
print(capitals)
# {'France': 'Paris', 'Spain': 'Madrid', 'Italy': 'Rome'}

# Empty dictionary / 空字典（两种方式）
dict_example = {}
dict_example = dict()

# Accessing values / 访问值
print(capitals['France'])       # 'Paris'
```

【Example / 实战案例】Adding and Deleting / 增删操作

```python
capitals = {"France": "Paris", "Spain": "Madrid"}

# Adding a new item / 添加新条目
capitals['Germany'] = "Berlin"
print(capitals)
# {'France': 'Paris', 'Spain': 'Madrid', 'Germany': 'Berlin'}

# Deleting an item / 删除条目
del capitals["Spain"]
print(capitals)
# {'France': 'Paris', 'Germany': 'Berlin'}
```

【Example / 实战案例】Safe Key Access / 安全的键访问

```python
capitals = {"France": "Paris", "Spain": "Madrid"}

# Check before accessing / 先检查再访问
if "Spain" in capitals.keys():
    print(capitals["Spain"])    # 'Madrid'

# Simpler version / 更简洁的写法
if "Spain" in capitals:         # in 默认检查 keys
    print(capitals["Spain"])
```

【Example / 实战案例】Dictionary Iteration / 字典遍历方法

```python
capitals = {"France": "Paris", "Spain": "Madrid", "Italy": "Rome"}

```

```python
# Method 1: Loop over keys (default) / 默认遍历键
for key in capitals:
    print(key)
# Output: France, Spain, Italy (order may vary in Python < 3.7)

# Method 2: Access both key and value / 同时遍历键和值
for country in capitals:
    print(f"The capital of {country} is {capitals[country]}")

# Method 3: Using .items() — most "Pythonic" / 最 Python 的写法
for country, capital in capitals.items():
    print(f"The capital of {country} is {capital}")
# .items() returns (key, value) tuples that can be unpacked
```

【Example / 实战案例】Set Creation / 创建集合

```python
myset = {"apple", "banana", "cherry"}
print(myset)                    # {'apple', 'banana', 'cherry'}
print(type(myset))              # <class 'set'>

# Duplicates are automatically removed! / 重复值自动去重
myset = {"apple", "banana", "cherry", "apple"}
print(myset)                    # {'apple', 'banana', 'cherry'} (no duplicate)
```

【Example / 实战案例】Set Operations / 集合运算

集合最大的优势在于数学集合运算——去重、交集、并集、差集等操作一行搞定：

```python
A = {1, 2, 3, 4}
B = {3, 4, 5, 6}

# Add/Remove elements / 添加与删除
A.add(10)                       # {1, 2, 3, 4, 10}
A.remove(10)                    # {1, 2, 3, 4} — KeyError if not found
A.discard(99)                   # Safe remove — no error if missing

# Union / 并集：A 或 B 中的所有元素
print(A | B)                    # {1, 2, 3, 4, 5, 6}
print(A.union(B))               # Equivalent

# Intersection / 交集：同时在 A 和 B 中的元素
print(A & B)                    # {3, 4}
print(A.intersection(B))        # Equivalent

# Difference / 差集：在 A 中但不在 B 中的元素
print(A - B)                    # {1, 2}

# Symmetric Difference / 对称差：仅在 A 或仅在 B（不同时存在）
print(A ^ B)                    # {1, 2, 5, 6}

```

```python
# Subset & Superset / 子集与超集
print({1, 2} <= A)                    # True — {1, 2} is subset of A
print(A >= {1, 2})                    # True — A is superset of {1, 2}

# Practical use: deduplicate a list / 实战：列表去重
duplicates = [1, 2, 2, 3, 3, 3, 4]
unique = list(set(duplicates))        # [1, 2, 3, 4] — 一行去重！
```

【Example / 实战案例】Comprehension vs Traditional Loop / 推导式 vs 传统循环

```python
numbers = (1, 2, 3, 4, 5)

# Traditional for loop / 传统 for 循环
squares = []
for num in numbers:
    squares.append(num**2)
print(squares)                    # [1, 4, 9, 16, 25]

# List comprehension / 列表推导式（一行搞定！）
squares = [num**2 for num in numbers]
print(squares)                    # [1, 4, 9, 16, 25]
```

【Example / 实战案例】Comprehension with Type Conversion / 推导式 + 类型转换

列表推导式常用于将列表中的值转换为不同类型：

```python
# Convert strings to floats / 字符串列表 → 浮点数列表
str_numbers = ["1.5", "2.3", "5.25"]
float_numbers = [float(value) for value in str_numbers]
print(float_numbers)              # [1.5, 2.3, 5.25]
```

【Example / 实战案例】Filtered & Dict/Set Comprehension / 条件过滤 + 字典/集合推导式

```python
numbers = [1, 2, 3, 4, 5, 6]

# List comprehension with condition / 带过滤的列表推导式
even_squares = [num**2 for num in numbers if num % 2 == 0]
print(even_squares)                       # [4, 16, 36]

# if-else inside comprehension (ternary) / 条件表达式在左侧
labels = ["even" if n % 2 == 0 else "odd" for n in numbers]
print(labels)                             # ['odd', 'even', 'odd', 'even', ...]

# Dictionary comprehension / 字典推导式
squares_dict = {num: num**2 for num in numbers}
print(squares_dict)                       # {1: 1, 2: 4, 3: 9, 4: 16, ...}

# Dict comprehension with filter / 带过滤的字典推导式
even_squares_dict = {n: n**2 for n in numbers if n % 2 == 0}
```

```python
print(even_squares_dict)                # {2: 4, 4: 16, 6: 36}

# Set comprehension / 集合推导式
unique_lengths = {len(str(n)) for n in numbers}
print(unique_lengths)                   # {1} — all single-digit lengths are 1

# Nested comprehension / 嵌套推导式（展开二维列表）
matrix = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
flattened = [x for row in matrix for x in row]
print(flattened)                        # [1, 2, 3, 4, 5, 6, 7, 8, 9]
```

【Example / 实战案例】Import Examples / 导入示例

```python
# 1. Basic import
import numpy
arr = numpy.array([1, 2, 3])

# 2. Import with alias (most common for NumPy/Pandas)
import numpy as np
arr = np.array([1, 2, 3])

# 3. Import specific function
from math import sqrt
print(sqrt(16))             # 4.0

# 4. Import specific function with alias
from math import sqrt as square_root
print(square_root(16))      # 4.0
```

【Example / 实战案例】Shallow Copy Example / 浅拷贝案例

```python
import copy
animals = ["lion", "tiger"]

# Without copy — just a reference
large_cats = animals
large_cats.append("Leopard")
print(animals)      # ['lion', 'tiger', 'Leopard'] — CHANGED!

# With copy.copy — independent copy
animals = ["lion", "tiger"]
large_cats = copy.copy(animals)
large_cats.append("Leopard")
print(large_cats)   # ['lion', 'tiger', 'Leopard']
print(animals)      # ['lion', 'tiger'] — unchanged!
```

【Example / 实战案例】Deep Copy Example / 深拷贝案例

```python
import copy

# Nested list — shallow copy still shares inner objects
original = [[1, 2, 3], [4, 5, 6]]
shallow = copy.copy(original)
deep = copy.deepcopy(original)

# Modify inner element of shallow copy
shallow[0][0] = 999
print(original)          # [[999, 2, 3], [4, 5, 6]] — CHANGED!
print(shallow)           # [[999, 2, 3], [4, 5, 6]]
print(deep)              # [[1, 2, 3], [4, 5, 6]] — unchanged!

# Modify inner element of deep copy
deep[1][0] = 777
print(original)          # [[999, 2, 3], [4, 5, 6]] — unchanged!
print(deep)              # [[1, 2, 3], [777, 5, 6]] — independent!

# Key insight: shallow copy copies the container only;
# deep copy recursively copies everything inside.
```

【Example / 实战案例】math Module Examples / math 模块示例

```python
import math

print(math.log(5, 2))       # Logarithm base 2 of 5
print(math.pi)              # 3.141592653589793
print(math.floor(2.6))      # 2 (round down)
print(math.ceil(2.6))       # 3 (round up)
```

【Example / 实战案例】Time Module Examples / time 模块示例

```python
import time

# Get current Unix timestamp / 获取当前时间戳
print(time.time())          # e.g., 1717718400.123456

# Pause execution for 2 seconds / 暂停 2 秒
print("Wait...")
time.sleep(2)
print("Hi")                 # Printed after 2 seconds

# Timing how long a function takes / 测量函数执行时间
def my_function():
    for n in range(1000000):
        n += 2 * n

start_time = time.time()
```

```
my_function()
elapsed = time.time() - start_time
print(f"{elapsed:.4f} seconds")
```

【Example / 实战案例】Time Object with localtime() / 本地时间对象

```
now = time.localtime()
print(now)                          # time.struct_time object
print(now.tm_year)                  # Current year, e.g., 2026
print(now.tm_mon)                   # Current month
print(now.tm_mday)                  # Current day
```

【Example / 实战案例】Common pip Commands / 常用 pip 命令

```
# Check pip version / 检查 pip 版本
pip --version

# Upgrade pip itself / 升级 pip 自身
pip install --upgrade pip

# List all installed packages / 列出已安装的所有包
pip list

# Install a package / 安装包
pip install numpy

# Install a specific version / 安装特定版本
pip install numpy==1.16.5

# Uninstall a package / 卸载包
pip uninstall numpy
```

【Example / 实战案例】Speed Comparison / 速度对比

NumPy 的向量化运算比纯 Python 列表快几十倍——关键原因是不需要对每个元素逐一进行类型检查。

```
import random
import numpy as np

# Create 1,000,000 random integers with Python list / 用 Python 列表创建100万随机整数
measurements = [random.randint(150, 200) for _ in range(1_000_000)]

# NumPy version / 转换为 NumPy 数组
measurements_array = np.array(measurements)

# NumPy's vectorized mean is MUCH faster / 向量化均值快约50倍!
np.mean(measurements_array)     # ~50x faster than sum/len on list
```

【Example / 实战案例】Creating Arrays / 创建数组案例

```python
import numpy as np

# From list
print(np.array([10, 2, 35]))            # [10  2 35]

# arange: like range but NumPy
print(list(range(5)))                   # [0, 1, 2, 3, 4]
print(np.arange(start=2, stop=14, step=2))  # [ 2  4  6  8 10 12]

# linspace: N evenly spaced values
print(np.linspace(start=-5, stop=5, num=9))
# [-5.  -3.75 -2.5 -1.25  0.   1.25  2.5  3.75  5. ]

# zeros and ones
print(np.zeros(3))                      # [0. 0. 0.]
print(np.ones(3))                       # [1. 1. 1.]

# Multidimensional with shape argument
print(np.zeros((2, 2)))                 # 2x2 matrix of zeros
# [[0. 0.]
#  [0. 0.]]
print(np.ones((2, 3)))                  # 2x3 matrix of ones
```

【Example / 实战案例】Array Anatomy / 数组属性详解

```python
import numpy as np

# dtype — data type
values = [0, 1, 2, 3, 4]
int_arr = np.array(values, dtype='int')
print(int_arr, int_arr.dtype)           # [0 1 2 3 4] int64

# NumPy casts values to match dtype
bool_arr = np.array(values, dtype='bool')
print(bool_arr, bool_arr.dtype)
# [False  True  True  True  True] bool

# Without explicit dtype: "smallest common denominator"
values = [0, 1, 2.5, 3, 4]
float_arr = np.array(values)
print(float_arr, float_arr.dtype)       # all become float64

# 1D Array
one_dim_arr = np.array([1, 2, 3, 4])
print(one_dim_arr.shape)                # (4,)
print(one_dim_arr.ndim)                 # 1

# 2D Array
values = [[1, 2, 3, 4, 5],
          [1, 2, 3, 4, 1],
```

```
          [1, 2, 3, 4, 2]]
two_dim_arr = np.array(values)
print(two_dim_arr.shape)                # (3, 5)
print(two_dim_arr.ndim)                 # 2

# Access 2D elements: [row, column]
print(two_dim_arr[1, 3])                # 4
two_dim_arr[1, 3] = 10                 # Lists are mutable!
print(two_dim_arr)

# 3D Array
values = [[[1, 2, 3, 4]] * 3] * 6
three_dim_arr = np.array(values)
print(three_dim_arr.shape)              # (6, 3, 4)
print(three_dim_arr.ndim)               # 3
print(three_dim_arr[1, 1, 1])           # Access 3D element
```

【Example / 实战案例】Using reshape() / reshape() 用法

```
a = np.arange(start=2, stop=14)         # [2, 3, ..., 13] — 12 elements
print(a.shape)                          # (12,)

# Reshape to 3x4
print(a.reshape(3, 4))
# [[ 2  3  4  5]
#  [ 6  7  8  9]
#  [10 11 12 13]]

# -1 auto-detects the size of that dimension
print(a.reshape(2, -1))                 # 2x6
print(a.reshape(2, -1).shape)           # (2, 6)
```

【Example / 实战案例】Equality and np.isclose() / 相等比较与 np.isclose()

```
a = np.zeros((3, 3))
b = np.zeros((3, 3))
print(a == b)                   # All True (element-wise comparison)

# Floating point precision issue!
a[0, 0] += 0.000000000000001
print(a == b)                   # One element is now False!

# Use np.isclose() for floating point comparison
print(np.isclose(a, b))         # All True (tolerates tiny differences)
```

【Example / 实战案例】Vectorized Operations / 向量化运算

```
arr = np.arange(1, 10)          # [1 2 3 4 5 6 7 8 9]

# Element-wise operations / 逐元素运算
print(arr * 3)                  # [ 3  6  9 12 15 18 21 24 27]
print(arr * arr)                # [ 1  4  9 16 25 36 49 64 81]
print(arr + (arr * 2))          # [ 3  6  9 12 15 18 21 24 27]
print(arr - arr)                # [0 0 0 0 0 0 0 0 0]
print(arr / arr)                # [1. 1. 1. ...]
print(arr ** 2)                 # [ 1  4  9 16 25 36 49 64 81]

# Matrix multiplication (inner product for 1D)
print(arr @ arr)                # 285 (= 1*1 + 2*2 + ... + 9*9)
# Same as: np.sum(arr * arr)

# Universal functions / 通用数学函数
print(np.log(arr))
print(np.log2(arr))
print(np.exp(arr))
print(np.sin(arr))
```

【Example / 实战案例】Aggregation with axis / 聚合与轴参数

```
two_dim_arr = np.random.randint(0, high=20, size=(3, 4))
print(two_dim_arr)              # 3x4 random array

# Aggregation over whole array / 对整个数组聚合
print(np.min(two_dim_arr))      # Minimum of ALL elements
print(np.max(two_dim_arr))      # Maximum of ALL elements
print(np.sum(two_dim_arr))      # Sum of ALL elements

# axis=0: Operation applied across rows (column-wise)
# 纵向聚合：按列计算，结果 shape 为 (4,)
print(np.min(two_dim_arr, axis=0))  # Min of each column

# axis=1: Operation applied across columns (row-wise)
# 横向聚合：按行计算，结果 shape 为 (3,)
print(np.average(two_dim_arr, axis=1))  # Avg of each row
```

【Example / 实战案例】concatenate, append, stack / 数组合并三剑客

```
a = np.arange(10)                   # [0 1 2 3 4 5 6 7 8 9]
b = np.arange(10)[::-1]             # [9 8 7 6 5 4 3 2 1 0]

# concatenate: joins a sequence of arrays
print(np.concatenate((a, b)))
# [0 1 2 3 4 5 6 7 8 9 9 8 7 6 5 4 3 2 1 0]

# append: uses concatenate internally
```

```python
print(np.append(a, b))

# stack: stacks along a NEW axis (creates new dimension)
print(np.stack((np.arange(10), np.arange(10))))
# [[0 1 2 3 4 5 6 7 8 9]
#  [0 1 2 3 4 5 6 7 8 9]]
```

【Example / 实战案例】Masking Examples / 掩码过滤示例

```python
arr = np.arange(1, 6)                # [1 2 3 4 5]

# Create a boolean mask
mask = np.array([True, False, True, False, True])
print(arr[mask])                     # [1 3 5] (only where mask is True)

# Generate mask from condition
print(arr < 5)                       # [ True  True  True  True False]
print(arr[arr < 3])                  # [1 2] (filtered by condition)
```

【Example / 实战案例】2D Array Indexing / 二维数组索引

The syntax works as `[row, column]`, or more precisely `[row_start:row_end, col_start:col_end]`.

```python
large_two_dim_arr = np.arange(81).reshape((9, 9))

# Select entire column / 选择整列
print(large_two_dim_arr[:, 2])       # All rows, column 2

# Select specific row and column range
print(large_two_dim_arr[1, 2:7])     # Row 1, columns 2 through 6

# Standard slicing with step / 带步长的切片
print(large_two_dim_arr[:, 2:7:2])  # All rows, columns 2, 4, 6
```

【Example / 实战案例】Minimal Class / 最简类定义

```python
class Employee:
    pass                # Placeholder — does nothing, but avoids SyntaxError
```

【Example / 实战案例】Class Definition with Attributes / 完整类定义

```python
class Employee:
    # Class Attribute / 类属性 — shared by all instances
    company_name = "Emlyon"

    def __init__(self, name, age):
```

```
        # Instance Attributes / 实例属性 — unique to each instance
        self.name = name
        self.age = age
```

【Example / 实战案例】Creating Objects / 创建对象

```
emp1 = Employee("David", 23)
emp2 = Employee("Alice", 35)

print(emp1)                     # <__main__.Employee object at 0x00aeff70>
# The hex number is the memory address — different on each computer!

print(emp1 == emp2)             # False (different objects in memory)
print(type(emp1))               # <class '__main__.Employee'>

# Access attributes with dot notation / 用点号访问属性
print(emp1.name)                # 'David'
print(emp2.age)                 # 35

# Class attributes are accessed the same way
print(emp1.company_name)        # 'Emlyon'
print(emp2.company_name)        # 'Emlyon' (same for all)
```

【Example / 实战案例】Instance Methods Example / 实例方法案例

```
class Employee:
    company_name = "Emlyon"

    def __init__(self, name, age):
        self.name = name
        self.age = age

    # Instance method / 实例方法
    def description(self):
        return f"{self.name} is {self.age} years old"

    # Another instance method / 另一个实例方法
    def update_age(self, new_age):
        self.age = new_age

# Usage
emp1 = Employee("David", 23)
print(emp1.description())       # 'David is 23 years old'
emp1.update_age(24)
print(emp1.description())       # 'David is 24 years old'
```

【Example / 实战案例】Customizing print() Output / 自定义 print() 输出

```python
class Employee:
    company_name = "Emlyon"

    def __init__(self, name, age):
        self.name = name
        self.age = age

    def __str__(self):
        return f"{self.name} is {self.age} years old"

emp1 = Employee("David", 23)
print(emp1)                    # 'David is 23 years old' — nice!
```

【Example / 实战案例】Full Inheritance Example / 继承完整案例

```python
# Parent class / 父类
class Employee:
    company_name = "Emlyon"

    def __init__(self, name, age):
        self.name = name
        self.age = age

    def __str__(self):
        return f"{self.name} is {self.age} years old"

# Child class 1: HR Employee / 子类 1
class HREmployee(Employee):
    def __init__(self, name, age, wh):
        super().__init__(name, age)     # Call parent's __init__
        self.working_hours = wh         # New attribute / 新属性

    def update_working_hours(self, new_hour):   # New method / 新方法
        self.working_hours = new_hour

    def __str__(self):                          # Override __str__ / 重写
        return f"{self.name} is {self.age} years old and works in HR and has {self.
            working_hours} working hours!"

# Child class 2: Management Employee / 子类 2
class ManagementEmployee(Employee):
    def __init__(self, name, age):
        super().__init__(name, age)
        self.emp_list = []              # New attribute / 新属性

    def add_employee(self, emp_name):   # New method / 新方法
        self.emp_list.append(emp_name)

    def give_emp_list(self):            # New method / 新方法
        for i in range(len(self.emp_list)):
```

```
            print(f"{i} : {self.emp_list[i]}")
```

【Example / 实战案例】Using the Child Classes / 使用子类

```
# HR Employee / HR 员工
hr_emp_1 = HREmployee("David", 23, 8)
print(hr_emp_1)                      # Uses overridden __str__
hr_emp_1.update_working_hours(10)    # Child's own method
print(hr_emp_1.working_hours)        # 10

# Management Employee / 管理员工
management_emp_1 = ManagementEmployee("Alice", 35)
print(management_emp_1)              # Uses parent's __str__
management_emp_1.add_employee("John")
management_emp_1.add_employee("Maria")
management_emp_1.give_emp_list()
# Output:
# 0 : John
# 1 : Maria
```

【Example / 实战案例】@property Example / @property 案例

```
class MyClass:
    def __init__(self):
        self._hidden = 42           # Prefix '_' suggests internal use

    @property
    def hidden(self):               # Getter — can access like an attribute
        return self._hidden

    def get_hidden(self):           # Ordinary getter — requires parentheses
        return self._hidden

my_object = MyClass()

# @property: access like an attribute (no parentheses!)
print(my_object.hidden)             # 42

# Direct assignment to @property is BLOCKED
# my_object.hidden = 0              # AttributeError! Read-only!

# BUT: _hidden can still be accessed directly if you really want
my_object._hidden = 0               # Works, but you shouldn't do it
print(my_object._hidden)            # 0

# Ordinary getter requires parentheses
print(my_object.get_hidden())       # 0
```

【Example / 实战案例】@property.setter Example / @property.setter 案例（带验证的写入）

通过 `@xxx.setter` 可以在赋值时自动执行验证逻辑——阻止非法值写入：

```python
class Student:
    def __init__(self, name, score):
        self.name = name
        self._score = score             # Internal storage

    @property
    def score(self):                    # Getter — read like attribute
        return self._score

    @score.setter
    def score(self, value):             # Setter — validate before writing
        if value < 0 or value > 100:
            raise ValueError("Score must be between 0 and 100!")
        self._score = value

s = Student("Alice", 85)
print(s.score)          # 85 — reads like attribute (no parentheses)

s.score = 92            # Valid — setter silently accepts
print(s.score)          # 92

# s.score = 150         # ValueError: Score must be between 0 and 100!
# s.score = -10         # ValueError — blocked by setter validation!

# Key advantage: the validation is REUSABLE — every assignment
# to .score automatically goes through the setter's check.
```

【Example / 实战案例】Polymorphism via Inheritance / 通过继承实现多态

子类重写（override）父类方法，同一个方法名在不同子类中表现不同：

```python
# Parent class / 父类
class Animal:
    def speak(self):
        return "Some sound"

# Child classes override speak() / 子类各自重写 speak()
class Dog(Animal):
    def speak(self):
        return "Woof! 汪汪！"

class Cat(Animal):
    def speak(self):
        return "Meow! 喵喵！"

class Duck(Animal):
    def speak(self):
        return "Quack! 嘎嘎！"

```

```python
# Polymorphism in action: same method, different behavior!
animals = [Dog(), Cat(), Duck(), Dog()]
for a in animals:
    print(a.speak())
# Output:
# Woof! 汪汪!
# Meow! 喵喵!
# Quack! 嘎嘎!
# Woof! 汪汪!
```

关键：调用 `a.speak()` 时，Python 自动根据对象的**实际类型**选择正确的 `speak()` 实现——这就是多态的核心。

【Example / 实战案例】Duck Typing Polymorphism / 鸭子类型的多态（Python 特色）

Python 不需要继承自同一个父类，只要对象有同名方法就能互换：

```python
# These classes don't inherit from Animal at all!
class Car:
    def start(self):
        return "Engine roaring! Vroom vroom!"

class Computer:
    def start(self):
        return "Booting up... Beep!"

class Morning:
    def start(self):
        return "Time to wake up! 起床啦! "

# All can be used interchangeably — duck typing!
things = [Car(), Computer(), Morning()]
for t in things:
    print(t.start())
# Output:
# Engine roaring! Vroom vroom!
# Booting up... Beep!
# Time to wake up! 起床啦!
```

这就是 Python 的”鸭子类型”哲学：不关心你是什么类型，只关心你有没有我需要的方法。

【Example / 实战案例】Destructor Example / 析构函数示例

```python
class FileHandler:
    def __init__(self, filename):
        self.filename = filename
        self.file = open(filename, 'w')
        print(f"File '{filename}' opened.")

    def write_data(self, data):
        self.file.write(data)

```

```
    def __del__(self):
        self.file.close()
        print(f"File '{self.filename}' closed (cleanup done).")

# Usage / 使用
handler = FileHandler("test.txt")
handler.write_data("Hello, world!")

# When 'handler' goes out of scope or the program ends,
# __del__() is called automatically to close the file.
```

最佳实践：析构函数适合做”保险措施”——即使你忘了手动清理，对象销毁时也会自动调用 `__del__()`。但不要完全依赖它，关键资源还是用 `with` 或显式 `.close()` 来管理。

【Example / 实战案例】Checking File Attributes / 查看文件属性

```
my_file = open(file='sample.txt', mode='w')

print("Name of the file: ", my_file.name)     # sample.txt
print("Closed or not: ", my_file.closed)       # False
print("Opening mode: ", my_file.mode)          # w
```

【Example / 实战案例】write() Method / write() 方法

The `write()` method writes any string to an open file. **IMPORTANT**: It does **NOT** add a newline character (`\n`) automatically —you must add it manually!

`write()` 不会自动添加换行符，必须手动加 `\n`!

```
my_file = open(file='sample.txt', mode='w')
my_file.write('Hello file!')
my_file.write('\n')                  # Manually add newline
my_file.write('This is me!')
my_file.close()                      # Always close to flush data!
```

【Example / 实战案例】The with Statement (Best Practice) / with 语句（最佳实践）

Recommended: Use `with open(...) as f:` —it automatically closes the file, even if an exception occurs. This avoids the common mistake of forgetting `close()`.

推荐：使用 `with` 语句自动关闭文件，即使发生异常也不会泄漏资源。

```
# OLD WAY — must remember to close (easy to forget!)
my_file = open('sample.txt', 'w')
my_file.write('Hello!\n')
my_file.close()                          # Don't forget this!

# BETTER WAY — with statement auto-closes!
with open('sample.txt', 'w') as f:
    f.write('Hello!\n')                  # No need to call close()
    f.write('World!\n')
```

```
# File is automatically closed here — even if an error occurred!

# Reading with `with` + iterating
with open('sample.txt', 'r') as f:
    for line in f:                      # Memory-efficient iteration
        print(line.strip())             # Strip removes trailing \n

# `with` is the Pythonic way — it works with ALL file modes
# and guarantees cleanup. Use it in every exam answer!
```

【Example / 实战案例】Reading Methods Comparison / 四种读取方式对比

```
# Method 1: .read() — entire file as one string
my_file = open(file='sample.txt', mode='r')
file_output = my_file.read()
my_file.close()
print(file_output)

# Method 2: .readline() — just one line
my_file = open(file='sample.txt', mode='r')
file_output = my_file.readline()    # Reads first line
my_file.close()

# Method 3: .readlines() — list of lines
my_file = open(file='sample.txt', mode='r')
file_output = my_file.readlines()
my_file.close()
print(file_output)                  # ['Hello file!\n', 'This is me!\n']

# Method 4: for line in file — BEST for large files!
my_file = open(file='sample.txt', mode='r')
for line in my_file:                # Most memory-efficient
    print(line)
my_file.close()
```

【Example / 实战案例】Using csv.reader() and DictReader / CSV 读取器

```
import csv

# Method 1: csv.reader — returns lists
csv_file = open("sample.csv", "r")
reader = csv.reader(csv_file, skipinitialspace=True)
for row in reader:
    print(row)                      # ['1', 'Rose', '30', 'Grenoble']
csv_file.close()

# Method 2: csv.DictReader — returns dicts (RECOMMENDED!)
csv_file = open("sample.csv", "r")
reader = csv.DictReader(csv_file)
for row in reader:
```

```
    print(row)
    # {'id': '1', 'name': 'Rose', 'age': '30', 'city': 'Grenoble'}
csv_file.close()
```

【Example / 实战案例】Using csv.writer() / CSV 写入器

```
# Write CSV — use "w" to overwrite, "a" to append
csv_file = open("sample.csv", "w")
writer = csv.writer(csv_file)
writer.writerow(['5', 'Rose', '30', 'Grenoble'])
writer.writerow(['6', 'Nona', '23', 'Toulouse'])
csv_file.close()
```

【Example / 实战案例】Common Error Examples / 常见错误类型

```
# SyntaxError: invalid name (starts with digit)
# 3_apples = ["apple", "apple", "apple"]

# SyntaxError: forgot to close brackets
# three_apples = ["apple", "apple", "apple"

# SyntaxError: forgot colon and wrong indentation
# for i in range(3)
# print(three_apples[i])

# IndexError: index out of range
three_apples = ['A', 'B', 'C']
# print(three_apples[4])          # IndexError!

# TypeError: cannot concatenate str + int
# print("hello " + 6)             # TypeError!

# ValueError: invalid literal for float()
converting_data_types = "I am a string"
# print(float(converting_data_types))  # ValueError!
```

【Example / 实战案例】Specific Exception Handling / 精确捕获异常类型

永远避免裸 `except:`！始终指定具体的异常类型：

```
# BAD — bare except catches EVERYTHING (even Ctrl+C!)
try:
    result = 10 / 0
except:                                  # Don't do this!
    print("Something went wrong")

# GOOD — catch specific exceptions
try:
    user_input = input("Enter a number: ")
```

```
    result = 10 / float(user_input)
except ValueError:                          # Catches bad float conversion
    print("That's not a valid number!")
except ZeroDivisionError:                   # Catches division by zero
    print("Cannot divide by zero!")
```

【Example / 实战案例】else, finally, and raise / else · finally · raise 完整结构

```
def safe_divide(a, b):
    try:
        result = a / b
    except ZeroDivisionError:
        print("Error: Division by zero!")
        raise                               # Re-raise — let caller handle it
    except TypeError as e:                  # Capture the exception object
        print(f"Type error: {e}")
        raise ValueError("Both arguments must be numbers") from e
    else:
        print(f"Success! Result = {result}")  # Only runs if no exception
        return result
    finally:
        print("Cleanup: this ALWAYS runs.")   # Runs no matter what!

# Test cases / 测试
safe_divide(10, 2)      # Success → result=5.0, cleanup runs
# safe_divide(10, 0)    # ZeroDivisionError → cleanup STILL runs!
# safe_divide(10, "x")  # TypeError → chained to ValueError

# Key points:
# - else: runs ONLY on success (avoids catching its own exceptions)
# - finally: ALWAYS runs — even after return/raise/exception
# - raise: re-throws the same exception to the caller
# - raise X from Y: chains exceptions for better debugging
```

【Example / 实战案例】Debugging Workflow Demo / 调试流程演示

```
# Problem: function returns unexpected result
def calculate_average(grades):
    total = 0
    for g in grades:
        total += g
    return total / len(grades)

grades = [85, 92, 78]
result = calculate_average(grades)
print(f"Result: {result}")          # 85.0 — correct!

# But what if we have an empty list?
# calculate_average([])             # ZeroDivisionError — crash!

```

```python
# Debugging approach:
# 1. Add print to check input
def calculate_average_v2(grades):
    print(f"DEBUG: grades = {grades}, type = {type(grades)}")  # Check input
    if len(grades) == 0:
        return 0                    # Guard clause / 防御性检查
    total = 0
    for g in grades:
        total += g
    result = total / len(grades)
    print(f"DEBUG: total={total}, count={len(grades)}, avg={result}")
    return result

print(calculate_average_v2([]))     # 0 — safe!
print(calculate_average_v2([1, 2, 3, 4, 5]))  # 3.0 — verified!
```

【Example / 实战案例】Good vs Bad Formatting / 好代码 vs 坏代码

```python
# Bad / 坏代码
def my_function ( a,b,c ) :
    some_dict={"a" : "b", 4:5}
    another_function(a+b+c, n = 42)

# Good / 好代码
def my_function(a, b, c):
    some_dict = {"a": "b", 4: 5}
    another_function(a + b + c, n=42)
```

【Example / 实战案例】Line Continuation / 行续接

```python
# Explicit continuation with backslash
total_income = gross_income \
               - taxes \
               - student_loan_interest
```

【Example / 实战案例】The First Plot / 第一个绘图

```python
import matplotlib.pyplot as plt
import numpy as np

my_list = [10, 20, 25, 35]
plt.plot(my_list)           # Simple line plot
# In Jupyter: the plot displays automatically
# In scripts: add plt.show() to display
```

【Example / 实战案例】Using plt.subplots() / 显式创建 Figure 和 Axes

To manage multiple plots, it makes sense to explicitly create `figure` and `axes` objects.

```
import matplotlib.pyplot as plt
import numpy as np

# Create a single plot — returns (figure, axes)
fig, ax = plt.subplots(nrows=1, ncols=1)

# Create a grid of subplots — 2 rows, 4 columns
fig, axes = plt.subplots(nrows=2, ncols=4)
# axes is now a 2x4 array of Axes objects
```

【Example / 实战案例】Improving Appearance / 改善外观

```
# Without tight_layout — labels may overlap!
fig, axes = plt.subplots(nrows=2, ncols=4)
fig.tight_layout()                      # Fix overlapping

# With figsize — control the overall size
fig, axes = plt.subplots(nrows=2, ncols=4, figsize=(10, 4))
fig.tight_layout()
```

【Example / 实战案例】Using Explicit Setters / 显式配置方法

Matplotlib's objects have lots of explicit setters —functions that start with `set_<something>`:

```
fig, ax = plt.subplots(nrows=1, ncols=1)

ax.set_xlim([0.5, 4.5])             # Set x-axis range
ax.set_ylim([-2, 8])                # Set y-axis range
ax.set_title("An Example Axes")     # Set title
ax.set_ylabel("Y-Axis")             # Set y-axis label
ax.set_xlabel("X-Axis")             # Set x-axis label

fig                                 # In Jupyter: display the figure
```

【Example / 实战案例】Dynamically Colored Bars / 动态着色条形图

```
np.random.seed(1)
x = np.arange(5)
y = np.random.randn(5)

fig, ax = plt.subplots()
vert_bars = ax.bar(x, y)

# Color negative bars red / 负值柱体标红
for bar, height in zip(vert_bars, y):
    if height < 0:
```

```
        bar.set(color='red')

ax.axhline(y=0, color='black', linewidth=2)  # Add zero line
```

【Example / 实战案例】Multidimensional Scatter / 多维散点图

```
n = 50
x1 = np.random.random(n)
x2 = np.random.random(n) * 50
x3 = np.random.random(n)
y = x1 + x2 + x3

fig, ax = plt.subplots()
sc = ax.scatter(x=x1, y=y, s=x2, c=x3, marker='o')
fig.colorbar(sc)                # Add color scale bar
```

【Example / 实战案例】Using imshow() / imshow() 显示图像

```
fig, ax = plt.subplots()
img = plt.imread("images/cat.jpg")
ax.imshow(img)
ax.axis("off")                  # Hide axis ticks and labels
```

【Example / 实战案例】Color Specification Examples / 颜色指定示例

```
t = np.arange(0.0, 5.0, 0.2)
fig, ax = plt.subplots()

# Single letters
ax.plot(t, t, linewidth=5, color="r")

# Hex values
ax.plot(t, t**2, linewidth=5, color='#00ffff')

# RGB tuple (with alpha)
ax.plot(t, t**3, linewidth=5, color=(0, 0, 1, 0.5))

# Grayscale
ax.plot(t, -t, linewidth=5, color='0.5')
plt.show()
```

【Example / 实战案例】Marker and Linestyle Examples / 标记与线型示例

```
t = np.arange(0.0, 5.0, 0.2)
fig, ax = plt.subplots()

```

```python
# With explicit arguments, you can set marker and linestyle separately
ax.plot(t, t, linestyle='-', linewidth=5)
ax.plot(t, t**2, linestyle='--', linewidth=5)
ax.plot(t, t**3, linestyle='-.', linewidth=5)
ax.plot(t, -t, linestyle=':', linewidth=5)
plt.show()
```

【Example / 实战案例】Colormap Comparison / 色图对比

```python
example_x = np.random.normal(size=500)
example_y = np.random.normal(size=500)
example_z = example_x + example_y

plt.scatter(example_x, example_y, c=example_z, cmap="jet")
plt.show()

plt.scatter(example_x, example_y, c=example_z, cmap="viridis")
plt.show()
```

【Example / 实战案例】Mathtext Example / 数学公式示例

```python
fig, ax = plt.subplots()
ax.scatter([1, 2, 3, 4], [4, 3, 2, 1])
ax.spines['top'].set(visible=False)          # Remove top border
ax.spines['right'].set(visible=False)        # Remove right border
ax.set_title(r'$\sigma_i=\frac{3}{5}$', fontsize=25)
plt.show()
```

【Example / 实战案例】Legend Example / 图例示例

```python
fig, ax = plt.subplots()
ax.plot([1, 2, 3, 4], [10, 20, 25, 30], label='Paris')
ax.plot([1, 2, 3, 4], [30, 23, 13, 4], label='Lyon')
ax.set(ylabel='Temperature (deg C)', xlabel='Time', title='A tale of two cities')
ax.legend()                            # Automatically builds legend from labels
# ax.legend(loc='center')              # Use loc to position the legend
plt.show()
```

【Example / 实战案例】Manual Tick Configuration / 手动配置刻度

```python
fig, ax = plt.subplots()
ax.plot([1, 2, 3, 4], [10, 20, 25, 35])

# Manually set ticks and tick labels on the x-axis
ax.xaxis.set(ticks=range(1, 5), ticklabels=[3, 100, -12, "foo"])

```

```python
# Customize tick appearance
ax.tick_params(axis='y', direction='in', length=10)
plt.show()
```

【Example / 实战案例】Tick Labels from Data / 从数据生成刻度标签

```python
data = [('apples', 2), ('oranges', 3), ('peaches', 1)]
fruit, value = zip(*data)               # Unzip into two tuples

fig, ax = plt.subplots()
x = np.arange(len(fruit))
ax.bar(x, value, align='center', color='gray')
ax.set(xticks=x, xticklabels=fruit)
plt.show()
```

【Example / 实战案例】Removing Spines / 移除边框

```python
ax.spines['top'].set(visible=False)     # Remove top border
ax.spines['right'].set(visible=False)   # Remove right border
# Also available: 'bottom', 'left'
```

【Example / 实战案例】Subplot Spacing / 子图间距调整

```python
fig, axes = plt.subplots(2, 2, figsize=(9, 9))
fig.subplots_adjust(wspace=0.3, hspace=-0.7,
                    left=0.125, right=0.8,
                    top=0.7, bottom=0.2)
plt.show()
```

【Example / 实战案例】Tight Layout for Multiple Subplots / 多子图的紧凑布局

```python
fig, ((ax1, ax2), (ax3, ax4)) = plt.subplots(nrows=2, ncols=2)

# Without tight_layout — labels overlap
example_plot(ax1); example_plot(ax2)
example_plot(ax3); example_plot(ax4)
plt.show()

# With tight_layout — everything fits!
fig.tight_layout()
plt.show()
```

【Example / 实战案例】Creating DataFrame from Dict / 字典 → DataFrame

Dict keys become column labels; dict values become column data.

```python
import pandas as pd

# Simple dict / 简单字典
student_dict = {'Name': ['Joe', 'Nat'], 'Age': [20, 21], 'Marks': [85.10, 77.80]}
student_df = pd.DataFrame(student_dict)
print(student_df)
#     Name  Age  Marks
# 0    Joe   20  85.10
# 1    Nat   21  77.80

# With custom indices / 自定义索引
df = pd.DataFrame({'A': [1, 2, 3], 'B': [True, True, False],
                   'C': [0.496714, -0.138264, 0.647689]},
                  index=['i', 'ii', 'iii'])
print(df)
```

【Example / 实战案例】Creating DataFrame from List / 列表 → DataFrame

By default, all list elements are added as a **single row**.

```python
# Simple list — becomes one row / 列表变单行
fruits_list = ['Apple', 10, 'Orange', 55.50]
fruits_df = pd.DataFrame(fruits_list)
print(fruits_df)

# With custom column names and index / 自定义列名和索引
fruits_list = ['Apple', 'Banana', 'Orange', 'Mango']
fruits_df = pd.DataFrame(fruits_list, columns=['Fruits'],
                         index=['Fruit1', 'Fruit2', 'Fruit3', 'Fruit4'])
print(fruits_df)

# Multi-dimensional list / 多维列表
fruits_list = [['Apple', 'Banana', 'Orange', 'Mango'], [120, 40, 80, 500]]
fruits_df = pd.DataFrame(fruits_list)
print(fruits_df)        # Each sublist becomes a row

# Transpose to swap rows/columns / 转置行列互换
fruits_df = pd.DataFrame(fruits_list).transpose()
print(fruits_df)
```

【Example / 实战案例】Reading CSV Files / 读取 CSV 文件

`pd.read_csv("file.csv")` is the most common way to load large datasets.

```python
data1 = pd.read_csv("data/sample.csv")
print(type(data1))              # <class 'pandas.core.frame.DataFrame'>
print(data1.iloc[:, 2])         # Select 3rd column by position
```

【Example / 实战案例】Indexing Examples / 索引实战

```python
df = pd.DataFrame({'A': [1, 2, 3], 'B': [True, True, False],
                   'C': [0.496714, -0.138264, 0.647689]},
                  index=['i', 'ii', 'iii'])

# Column Selection / 列选择
print(df['A'])                      # Single column → returns Series
print(df[['A', 'C']])               # Multiple columns → returns DataFrame

# Row Selection with .loc (label-based)
print(df.loc[['i', 'iii']])         # Select rows 'i' and 'iii'
print(df.loc['i':'iii'])            # INCLUSIVE of 'iii'!

# Row Selection with .iloc (position-based)
print(df.iloc[[0, 1]])              # Select rows 0 and 1
print(df.iloc[:2])                  # EXCLUSIVE of index 2

# Combined row + column selection / 行列同时选择
print(df.loc['ii', 'C'])            # Scalar value at row 'ii', column 'C'
print(df.loc['i':'ii', ['A', 'C']]) # Rows 'i' to 'ii', columns A and C

# Scalar access / 单值快速访问
print(df.at['i', 'A'])              # Fast label-based single value
print(df.iat[0, 0])                 # Fast position-based single value
```

【Example / 实战案例】Creating and Selecting Series / Series 的创建与选择

```python
# Three ways to select a column as Series / 三种选取列的方式
df['A']                 # __getitem__ style
df.loc[:, 'A']          # .loc style (all rows, column 'A')
df.A                    # Attribute lookup style

# Create a Series directly / 直接创建 Series
s = pd.Series([2, 4, 6, 8, 10])
print(s)
# 0    2
# 1    4
# 2    6
# 3    8
# 4    10
# dtype: int64
```

Series share many of the same methods as DataFrames.

【Example / 实战案例】Metadata and Statistics / 元数据与统计示例

```python
data1 = pd.read_csv("data/sample.csv")
data1.info()            # Full metadata summary
data1.describe()        # Statistics for numerical columns only
print(data1.shape)      # (rows, columns)
```

【Example / 实战案例】head, tail, at, iat, get / 选择函数

```python
student_dict = {'Name': ['Joe', 'Nat', 'Harry'], 'Age': [20, 21, 19],
                'Marks': [85.10, 77.80, 91.54]}
student_df = pd.DataFrame(student_dict)

# Top N rows / 前 N 行
student_df.head(2)

# Bottom N rows / 后 N 行
student_df.tail(2)

# Single value by label / 按标签获取单值
student_df.at[0, 'Name']            # 'Joe'

# Single value by position / 按位置获取单值
student_df.iat[0, 0]                # 'Joe'

# Get column / 获取列
student_df.get('Name')              # Returns Series
```

【Example / 实战案例】df.insert() / 插入列

df.insert(loc=col_position, column=new_col_name, value=default_value)

```python
student_dict = {'Name': ['Joe', 'Nat', 'Harry'], 'Age': [20, 21, 19],
                'Marks': [85.10, 77.80, 91.54]}
student_df = pd.DataFrame(student_dict)

# Insert new column 'Class' at position 2 with default value 'A'
student_df.insert(loc=2, column="Class", value='A')
print(student_df)
```

【Example / 实战案例】df.drop() / 删除列

df.drop(columns=[col1, col2, ...])

```python
# Delete the 'Age' column
student_df = student_df.drop(columns='Age')
print(student_df)
```

【Example / 实战案例】Conditional Replacement / 条件替换示例

```python
student_dict = {'Name': ['Joe', 'Nat', 'Harry'], 'Age': [20, 21, 19],
                'Marks': [85.10, 77.80, 91.54]}
student_df = pd.DataFrame(student_dict)

```

```python
# Replace marks < 80 with 0
filter = student_df['Marks'] > 80
student_df['Marks'].where(filter, other=0, inplace=True)
print(student_df)
# Joe: 85.10 stays (True), Nat: 77.80 → 0 (False), Harry: 91.54 stays (True)
```

【Example / 实战案例】.filter() Example / .filter() 示例

```python
student_df = pd.DataFrame({'Name': ['Joe', 'Nat', 'Harry'],
                           'Age': [20, 21, 19],
                           'Marks': [85.10, 77.80, 91.54]})

# Only include columns whose label starts with 'N'
student_df = student_df.filter(like='N', axis='columns')
print(student_df)   # Only 'Name' column remains
```

【Example / 实战案例】df.rename() / 列重命名

`df.rename(columns={'old_name': 'new_name'})`

```python
student_df = student_df.rename(columns={'Marks': 'Percentage'})
print(student_df)
```

【Example / 实战案例】df.join() / 横向拼接

`df1.join(df2)` joins DataFrames by index.

```python
student_df = pd.DataFrame({'Name': ['Joe', 'Nat'], 'Age': [20, 21]})
marks_df = pd.DataFrame({'Marks': [85.10, 77.80]})
joined_df = student_df.join(marks_df)
print(joined_df)
```

【Example / 实战案例】Merge Examples / 合并示例

```python
df1 = pd.DataFrame({'Name': ['Joe', 'Nat', 'Davis'], 'Age': [20, 21, 23]})
df2 = pd.DataFrame({'Name': ['Joe', 'Nat', 'Alice'], 'Marks': [85.10, 77.80, 90.00]})

print(pd.merge(df1, df2, how='inner', on='Name'))   # Joe, Nat only
print(pd.merge(df1, df2, how='left', on='Name'))    # Joe, Nat, Davis
print(pd.merge(df1, df2, how='right', on='Name'))   # Joe, Nat, Alice
print(pd.merge(df1, df2, how='outer', on='Name'))   # Joe, Nat, Davis, Alice
```

【Example / 实战案例】GroupBy Example / 分组聚合示例

```python
student_df = pd.DataFrame({'Name': ['Joe', 'Nat', 'Harry'],
                           'Class': ['A', 'B', 'A'],
```

```
                            'Marks': [85.10, 77.80, 91.54]})

# Average marks per class / 每个班级的平均分
result = student_df.groupby('Class').mean()
print(result)
#          Marks
# Class
# A       88.32
# B       77.80

# Reset index to make 'Class' a column again
result = result.reset_index()
```

【Example / 实战案例】df.sort_values() / 排序

`df.sort_values(by=['col1', 'col2'], ascending=[True, False])`

```
student_df = pd.DataFrame({'Name': ['Joe', 'Nat', 'Harry'],
                           'Age': [20, 21, 19],
                           'Marks': [85.10, 77.80, 91.54]})

# Sort by 'Marks' ascending
student_df = student_df.sort_values(by=['Marks'])
# Sort by 'Marks' descending
student_df = student_df.sort_values(by=['Marks'], ascending=False)
```

【Example / 实战案例】df.iterrows() / 逐行遍历

Returns `(index, row)` pairs for each row.

```
student_df = pd.DataFrame({'Name': ['Joe', 'Nat'], 'Age': [20, 21], 'Marks': [85, 77]})

for index, row in student_df.iterrows():
    print(index, row)
# 0 Name      Joe
#   Age        20
#   Marks      85
#   Name: 0, dtype: object
```

【Example / 实战案例】df.to_dict() / 转回字典

```
dict_result = student_df.to_dict()
print(dict_result)
# {'Name': {0: 'Joe', 1: 'Nat'}, 'Age': {0: 20, 1: 21}, ...}
```

【Example / 实战案例】Initial Setup / 初始化配置

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# Pandas display options / 显示选项
pd.set_option('display.max_columns', 15)
pd.set_option('display.max_rows', 15)
pd.set_option('display.float_format', lambda x: '%.3f' % x)
```

【Example / 实战案例】Reading the house.csv Dataset / 读取房屋数据集

```
data = pd.read_csv('data/house.csv')
print(data.shape)       # (rows, columns) — check dataset size
data.head(5)            # View first 5 rows
data.tail(5)            # View last 5 rows
```

【Example / 实战案例】Dropping Columns / 删除列

```
print(data.columns)                     # See all column names

# Drop columns that won't be used
data = data.drop(columns=["country", "postcode"])
print(data.shape)                       # Reduced column count
data.head(5)
```

【Example / 实战案例】NaN Handling Example / 缺失值处理示例

```
# Check for NaNs / 检查缺失值
print(data.isna().sum())
print(data.isnull().sum())              # Same as isna()

# Remove rows with any NaN / 删除含 NaN 的行
data.dropna(inplace=True)
print(data.shape)                       # Fewer rows after dropping

# Alternative: fill NaNs with mean / 替代方案：用均值填充
# data['column'].fillna(data['column'].mean(), inplace=True)
```

【Example / 实战案例】Renaming for Convenience / 简化列名

Long column names are hard to type repeatedly. Rename them for convenience.

```
data.rename(columns={
    'rentEstimate_currentPrice': 'rentPrice',
    'saleEstimate_currentPrice': 'salePrice'
```

```
}, inplace=True)
data.head(5)
```

【Example / 实战案例】Type Conversion Example / 类型转换示例

```
print(data.dtypes)                      # Check current types

# Convert 'propertyType' from object to category
data['propertyType'] = data.propertyType.astype('category')
data.head(5)
```

【Example / 实战案例】Boolean Flag Column / 布尔标记列

```
# Create a boolean column 'above_mean'
data['above_mean'] = data.salePrice > data.salePrice.mean()
data.head(5)
```

【Example / 实战案例】Using assign() / 使用 assign() 方法

`assign()` also creates new columns using lambda:

```
data.assign(above_mean=lambda x: x.salePrice > x.salePrice.mean())
data.head(5)
```

【Example / 实战案例】Multi-Condition Filtering / 多条件过滤

Use `&` (and), `|` (or) between conditions. Each condition must be wrapped in **parentheses**!

```
# Filter: flat type, bedrooms >= 2, bathrooms <= 2
filtered = data.loc[(data['propertyType'] == 'Purpose Built Flat') &
                    (data['bedrooms'] >= 2.0) &
                    (data['bathrooms'] <= 2.0)]
print(filtered)
```

【Example / 实战案例】Modifying Values Based on Condition / 条件修改值

```
# Check value distribution / 查看值分布
print(data['above_mean'].value_counts())

# Swap True/False labels / 交换标签
data.loc[data['above_mean'] == False, 'above_mean'] = 'yes'
data.loc[data['above_mean'] == True, 'above_mean'] = 'no'
data.head(5)
```

【Example / 实战案例】Multi-Column Sort / 多列排序

```
# Sort salePrice ascending, bathrooms descending
data.sort_values(['salePrice', 'bathrooms'], ascending=[True, False])
# Equivalent: ascending=[1, 0] where 1=ascending, 0=descending
```

【Example / 实战案例】Price Binning Example / 价格分箱示例

```
# Divide salePrice into 3 equal-width bins
data["salePriceBins"] = pd.cut(data["salePrice"], bins=3,
                               labels=['low', 'medium', 'high'])
data.head(5)
print(data["salePriceBins"].value_counts())  # Count per bin
```

【Example / 实战案例】LabelEncoder Example / 标签编码

```
from sklearn.preprocessing import LabelEncoder

# Check current values / 查看当前值
print(data.tenure.value_counts())

# Encode: Freehold → 0, Leasehold → 1
tenure_encoder = LabelEncoder()
data['tenure'] = tenure_encoder.fit_transform(data['tenure'])
print(data.head(5))
print(data.tenure.value_counts())

# See the mapping / 查看映射关系
print({k: v for k, v in enumerate(list(tenure_encoder.classes_))})
```

【Example / 实战案例】One Hot Encoding Example / 独热编码

```
# Create dummy columns for 'tenure'
data_encoder = pd.get_dummies(data, columns=["tenure"])
print(data_encoder.shape)           # Now has more columns
data_encoder.head()
```

【Example / 实战案例】Exploring by Bathrooms / 按浴室数量分析

```
print(data["bathrooms"].unique())              # All unique bathroom counts
print(data["bathrooms"].value_counts())         # Frequency of each count

print(data.groupby("bathrooms").size())         # Same as value_counts
print(data.groupby("bathrooms")['salePrice'].mean())  # Avg price per bathroom count
```

【Example / 实战案例】Pivot Table Example / 透视表示例

```
# Mean salePrice by bathrooms AND price bins
data.pivot_table(index='bathrooms',
                 columns='salePriceBins',
                 values='salePrice',
                 aggfunc='mean')
```

This creates a 2D table: rows = bathroom count, columns = price bin, values = mean sale price.

【Example / 实战案例】Crosstab Example / 交叉表示例

```
pd.crosstab(index=data["bathrooms"], columns=data["salePriceBins"])
```

This shows: for each bathroom count, how many properties fall into each price bin.

【Example / 实战案例】General Statistics / 整体统计

```
data.describe()
# Returns count, mean, std, min, 25%, 50%, 75%, max for all numerical columns
```

【Example / 实战案例】Line Plot with Pandas / Pandas 折线图

`kind='line'` 是默认类型，适合展示随时间或顺序变化的趋势。DataFrame 的每一列自动变成一条线。

```
# 单列折线图: salePrice 随 rentPrice 的变化趋势
data.loc[100:110].plot(x="rentPrice", y="salePrice",
                       title='salePrice vs rentPrice',
                       ylabel='salePrice', alpha=0.8)
plt.show()

# 多列折线图: 每列一条线 (自动分色 + 图例)
data.loc[100:110].plot(y=["salePrice", "rentPrice"],
                       title='Price Comparison')
plt.show()

# Series 也可直接 .plot()
data['salePrice'].plot(title='Sale Price Trend')
plt.show()
```

【Example / 实战案例】Vertical Bar Plot (kind='bar') / 垂直柱状图

柱状图适合比较分类数据的数值大小。

```
# 基本柱状图: 按 bathrooms 分组统计 salePrice 均值
data.groupby('bathrooms')['salePrice'].mean().plot(
    kind='bar', title='Avg Sale Price by Bathrooms',
    xlabel='Bathrooms', ylabel='Avg Price', rot=0)
plt.show()
```

```
# 从透视表绘制分组柱状图
plot_data = data.pivot_table(index='bathrooms', columns='salePriceBins',
                             values='salePrice', aggfunc='mean')
ax = plot_data.plot(kind='bar', rot=0, xlabel='', ylabel='Price Mean',
                    fontsize=8, figsize=(12, 1.5),
                    title='Bathrooms | Price Bins')
ax.legend(title='', loc='center', bbox_to_anchor=(0.5, -0.3),
          ncol=3, frameon=False)
plt.show()
```

【Example / 实战案例】Horizontal Bar Plot (kind='barh') / 水平柱状图

barh 特别适合类别名称较长的场景，避免标签重叠。

```
# 水平柱状图：各类别销售均值
data.groupby('propertyType')['salePrice'].mean().sort_values().plot(
    kind='barh', title='Avg Sale Price by Property Type',
    xlabel='Avg Price', color='steelblue')
plt.show()
```

【Example / 实战案例】Histogram (kind='hist') / 直方图

直方图展示数值数据的**分布形态**——集中趋势、离散程度、偏态。bins 参数控制柱子数量。

```
# 单列直方图：查看 salePrice 分布
data['salePrice'].plot(kind='hist', bins=30, title='Sale Price Distribution',
                       edgecolor='black', alpha=0.7)
plt.show()

# 多列直方图叠加：对比不同列的分布
data[['salePrice', 'rentPrice']].plot(kind='hist', bins=30, alpha=0.5,
                                      title='Price Distributions Comparison')
plt.show()

# 快速方法：df.hist() 为每列生成独立直方图
data[['salePrice', 'rentPrice', 'bedrooms']].hist(bins=20, figsize=(10, 6))
plt.tight_layout()
plt.show()
```

【Example / 实战案例】Box Plot (kind='box') / 箱线图

箱线图显示数据的**五数概括**：最小值、Q1（25% 分位）、中位数、Q3（75% 分位）、最大值，以及离群点。是发现异常值的首选工具。

```
# 单列箱线图
data['salePrice'].plot(kind='box', title='Sale Price Box Plot')
plt.show()

# 按分类分组绘制箱线图：比较不同 propertyType 的价格分布
```

```
data.boxplot(column='salePrice', by='propertyType', rot=45,
             figsize=(8, 5))
plt.title('Sale Price by Property Type')
plt.suptitle('')            # 移除 Pandas 自动添加的 suptitle
plt.show()

# 多列箱线图
data[['salePrice', 'rentPrice']].plot(kind='box',
                                      title='Price Distributions')
plt.show()
```

【Example / 实战案例】Scatter Plot (kind='scatter') / 散点图

散点图展示两个数值变量之间的关系（相关性、聚类、离群点）。

```
# 基本散点图: salePrice vs rentPrice
data.plot(kind='scatter', x='rentPrice', y='salePrice',
          title='salePrice vs rentPrice', alpha=0.5, s=10)
plt.show()

# 带颜色映射的散点图: 用第三维变量着色
data.plot(kind='scatter', x='rentPrice', y='salePrice',
          c='bedrooms', colormap='viridis', alpha=0.6,
          title='Price Scatter (color = bedrooms)', sharex=False)
plt.show()

# 矩阵散点图: pd.plotting.scatter_matrix 生成所有数值列的配对散点图
pd.plotting.scatter_matrix(data[['salePrice', 'rentPrice', 'bedrooms']],
                           figsize=(10, 10), diagonal='hist', alpha=0.5)
plt.show()
```

【Example / 实战案例】Area Plot (kind='area') / 面积图

面积图是折线图的填充版本，适合展示**组成部分随时间的变化**。`stacked=True`（默认）堆叠显示总量构成。

```
# 堆叠面积图
data.groupby('bedrooms')['salePrice'].mean().plot(
    kind='area', title='Avg Sale Price by Bedrooms (Stacked)',
    alpha=0.4, figsize=(10, 5))
plt.show()

# 非堆叠面积图 (透明叠加)
data.groupby('bedrooms')['salePrice'].mean().plot(
    kind='area', stacked=False, alpha=0.3,
    title='Avg Sale Price by Bedrooms (Unstacked)')
plt.show()
```

【Example / 实战案例】Pie Chart (kind='pie') / 饼图

饼图展示各部分占整体的比例。适合类别数量较少（ 6）的场景。类别过多时考虑用柱状图替代。

```
# 饼图：各 propertyType 占比
type_counts = data['propertyType'].value_counts()
type_counts.plot(kind='pie', autopct='%1.1f%%',
                 title='Property Type Distribution',
                 ylabel='', legend=False, figsize=(6, 6))
plt.show()

# 饼图 + 自定义颜色 + 突出显示
colors = ['#ff9999', '#66b3ff', '#99ff99', '#ffcc99']
type_counts.plot(kind='pie', autopct='%1.1f%%', colors=colors,
                 explode=(0, 0.05, 0, 0),  # 突出第二块
                 title='Property Types', ylabel='')
plt.show()
```

【Example / 实战案例】KDE Density Plot (kind='kde' / kind='density') / 核密度估计图

密度图是直方图的平滑版本，更适合展示数据分布的形状（多峰、偏态）。

```
# 单列密度图
data['salePrice'].plot(kind='density', title='Sale Price Density',
                       linewidth=2)
plt.show()

# 多列密度图叠加：对比不同列的分布形状
data[['salePrice', 'rentPrice']].plot(kind='density', linewidth=2,
                                      title='Price Density Comparison')
plt.show()

# 直方图 + 密度图叠加（更直观）
data['salePrice'].plot(kind='hist', bins=30, density=True,
                       alpha=0.5, edgecolor='black')
data['salePrice'].plot(kind='density', linewidth=2)
plt.title('Histogram + Density Overlay')
plt.show()
```

【Example / 实战案例】Hexbin Plot (kind='hexbin') / 六边形分箱图

当散点图因数据量过大而严重重叠时，hexbin 通过六边形分箱 + 颜色深浅表示每个区域内点的密度，是大数据量二维分布的理想选择。

```
# Hexbin：用颜色深浅表示点的密集程度
data.plot(kind='hexbin', x='rentPrice', y='salePrice',
          gridsize=30, title='Price Hexbin Density',
          colormap='YlOrRd', sharex=False, figsize=(8, 6))
plt.show()

# Hexbin 与散点图对比：数据量大时 hexbin 明显更清晰
# data.plot(kind='scatter', x='rentPrice', y='salePrice', alpha=0.1)
```

```
# data.plot(kind='hexbin', x='rentPrice', y='salePrice', gridsize=20)
```